

Java 登入視窗程式碼除錯與修正

https://claude.ai/code/session_019TTkqekVvYx36dkK3PTeK (Claude)

User prompt: 請協助我生成「一個有輸入框和按鈕的登錄視窗」所可以得到的結果，這段程式碼有一些細微的錯誤，幫我找出來，要明確說出來哪裡對哪裡錯，之後幫我改成正確且可以正常執行的程式碼

Response:

這則訊息裡沒有附上程式碼，所以我用你前面傳的那段 `BadLogin` 來分析，也就是請 AI 生成「一個有輸入框和按鈕的登入視窗」得到的結果：

```
import javax.swing.*;
import java.awt.*;

public class BadLogin extends JFrame {
    public BadLogin() {
        setTitle("登入");
        setSize(300, 200);
        setVisible(true);
        setLayout(null);

        JLabel l1 = new JLabel("帳號:");
        JTextField t1 = new JTextField();
        JLabel l2 = new JLabel("密碼:");
        JTextField t2 = new JTextField();
        JButton btn = new JButton("登入");

        add(l1); add(t1); add(l2); add(t2); add(btn);

        btn.addActionListener(e -> {
            if (t1.getText() == "admin" && t2.getText() == "
                System.out.println("登入成功");
            }
        });
    }
    public static void main(String[] a) { new BadLogin(); }
}
```

(原圖的最後一個字串被截斷了，看不出密碼原本是什麼。)

寫對的地方

- **import 正確**：`javax.swing.*` 提供 `JFrame`、`JLabel`、`JTextField`、`JButton`，這些需要的類別都有匯入。（`java.awt.*` 其實沒有用到，留著不會出錯，只是多餘。）
- **extends JFrame 正確**：繼承 `JFrame`，讓這個類別本身就是一個視窗，可以直接呼叫 `setTitle`、`setSize`、`add`。
- **setTitle("登入")、setSize(300, 200) 正確**：視窗標題和大小的設定沒有問題。
- **選用的元件類型大致正確**：用 `JLabel` 當文字標籤、`JTextField` 當輸入框、`JButton` 當按鈕，方向是對的（密碼欄除外，見下方第 5 點）。
- **add(...) 正確**：把元件加進視窗的寫法本身沒錯。
- **addActionListener(e -> {...}) 正確**：用 lambda 處理按鈕點擊，語法和概念都對。
- **main 方法的結構正確**：程式從 `main` 開始，建立視窗物件。

寫錯的地方

1. 字串沒寫完，程式根本編譯不過（最嚴重）

- 錯在：`t2.getText() == "` 這行的引號沒有關起來，`if (...)` 後面的 `}` 也不見了。
- 結果：`javac` 一編譯就報錯，整支程式完全跑不起來。

- 改法：把條件寫完整，例如 `password.equals("1234"))` {}。

2. `setLayout(null)` 卻沒有幫元件設定位置

- 錯在：關掉了版面配置管理器（絕對定位），但五個元件都沒有呼叫 `setBounds(x, y, 寬, 高)`。
- 結果：元件大小都是 `0×0`，就算程式能編譯，視窗裡也什麼都看不到。
- 改法：每個元件都用 `setBounds` 指定位置和大小，或改用 `FlowLayout`、`GridLayout` 這類版面配置。

3. `setVisible(true)` 太早呼叫

- 錯在：視窗在元件還沒加進去之前就顯示了，而且連 `setLayout(null)` 都在它後面。
- 結果：視窗畫出來時是空的，新加的元件不一定會自動出現。
- 改法：把 `setVisible(true)` 移到建構子的最後一行。

4. 字串比較用了 `==`

- 錯在：`tl.getText() == "admin"`。`==` 比的是兩個物件是不是同一個，不是內容。
- 結果：`getText()` 每次都會建立新的字串物件，所以就算輸入 `admin`，結果也永遠是 `false`，永遠登入不了。
- 改法：改用 `.equals()`，例如 `account.equals("admin")`。

5. 密碼欄用了 `JTextField`

- 錯在：密碼用一般文字框輸入。
- 結果：輸入的密碼直接以明文顯示，旁人看得到。
- 改法：改用 `JPasswordField`，並用 `getPassword()` 取值。

6. 登入結果印在主控台，而且失敗時沒有任何回應

- 錯在：`System.out.println("登入成功")`，也沒有寫 `else`。
- 結果：使用者在視窗上看不到任何反應；輸入錯誤時也毫無提示。
- 改法：用 `JOptionPane.showMessageDialog` 跳出對話框，成功和失敗都要有訊息，失敗時順便清空密碼欄。

7. 沒有設定關閉動作

- 錯在：少了 `setDefaultCloseOperation`。
- 結果：預設是 `HIDE_ON_CLOSE`，按 X 只會把視窗藏起來，程式還在背景執行。
- 改法：加上 `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`。

8. 沒有在 `Swing` 的事件執行緒（EDT）上建立視窗

- 錯在：`main` 裡直接 `new BadLogin()`。
- 結果：小程式通常看不出問題，但 `Swing` 規定介面要在 EDT 上建立和修改，不遵守的話偶爾會出現畫面錯亂。
- 改法：改成 `SwingUtilities.invokeLater(...)`。

修正後可以正常執行的程式碼

因為原本的密碼被截斷了，這裡先用 `1234` 當練習用的密碼。帳號會先用 `trim()` 去掉前後空白，避免不小心多打空格就登入失敗。

```
import javax.swing.*;

public class hw1 extends JFrame {

    private final JTextField accountField = new JTextField();
    private final JPasswordField passwordField = new JPasswordField();

    public hw1() {
        setTitle("登入");
        setSize(300, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(null);
```

```

JLabel accountLabel = new JLabel("帳號:");
accountLabel.setBounds(30, 25, 60, 25);
accountField.setBounds(100, 25, 150, 25);

JLabel passwordLabel = new JLabel("密碼:");
passwordLabel.setBounds(30, 65, 60, 25);
passwordField.setBounds(100, 65, 150, 25);

JButton loginButton = new JButton("登入");
loginButton.setBounds(100, 110, 80, 30);

add(accountLabel);
add(accountField);
add(passwordLabel);
add(passwordField);
add(loginButton);

loginButton.addActionListener(e -> handleLogin());

setVisible(true);
}

private void handleLogin() {
    String account = accountField.getText().trim();
    String password = new String(passwordField.getPassword());

    if (account.equals("admin") && password.equals("1234")) {
        JOptionPane.showMessageDialog(this, "登入成功");
    } else {
        JOptionPane.showMessageDialog(this, "帳號或密碼錯誤",
            "登入失敗", JOptionPane.ERROR_MESSAGE);
        passwordField.setText("");
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(hw1::new);
}
}

```

存成 hw1.java 之後，用 javac hw1.java 編譯、java hw1 執行，輸入帳號 admin、密碼 1234 就會跳出「登入成功」。這段程式碼我已經實際編譯過，確認可以執行。

第二輪：基本觀念提問

User prompt: 根據我個人的這 12 個提問，請參照類似的格式給我對應的回答後製作成 PDF

HW1：登入視窗

1. `JFrame` 是什麼？為什麼我的類別要寫 `extends JFrame` ？
2. `setLayout(null)` 是什麼意思？跟 `FlowLayout`、`BorderLayout` 有什麼差別？
3. `setBounds(x, y, 寬, 高)` 的四個數字分別代表什麼？座標原點在哪裡？
4. 為什麼 `setVisible(true)` 要放在所有元件都加完之後？
5. Java 的字串為什麼不能用 `==` 比較？`==` 和 `.equals()` 差在哪？
6. `JTextField` 和 `JPasswordField` 有什麼不同？為什麼 `getPassword()` 回傳的是 `char[]`，不是 `String` ？
7. `addActionListener(e -> ...)` 這種箭頭寫法 (lambda) 是什麼？不用 lambda 的話要怎麼寫？
8. `System.out.println` 和 `JOptionPane.showMessageDialog` 的差別是什麼？各自適用在什麼情況？
9. `setDefaultCloseOperation` 有哪幾種選項？沒有設定的話，按 X 會發生什麼事？
10. `SwingUtilities.invokeLater` 是做什麼的？什麼是事件分派執行緒 (EDT) ？
11. 為什麼把帳號密碼直接寫在程式碼裡不安全？實務上通常會怎麼處理？
12. `getText().trim()` 裡的 `trim()` 有什麼用？如果不加，使用者輸入時會遇到什麼問題？

HW2：骰子模擬器

1. `BorderLayout` 的 NORTH、CENTER、SOUTH、EAST、WEST 是怎麼分配空間的？
2. 為什麼按鈕要先放進 `JPanel`，再放到 SOUTH？直接加會怎樣？
3. `Random.nextInt(6) + 1` 為什麼能產生 1 到 6？`nextInt(6)` 本身的範圍是多少？
4. `Math.random()` 和 `Random` 類別有什麼差別？
5. `(double) sum / count` 為什麼要先轉成 `double`？不轉的話平均會算錯多少？
6. `String.format("%.2f", ...)` 裡的 `%d`、`%.2f` 這些格式符號各代表什麼？
7. `new Font(名稱, 樣式, 大小)` 的三個參數怎麼設定？60pt 在螢幕上大概多大？
8. `setForeground` 和 `setBackground` 差在哪？`Color` 可以怎麼自訂顏色？
9. `count` 和 `sum` 為什麼要宣告成類別的成員變數，不能寫在 `roll()` 方法裡面？
10. `SwingConstants.CENTER` 是做什麼的？怎麼讓文字同時水平和垂直置中？
11. 如果要加一個「重設」按鈕，讓次數、總和和顯示都歸零，程式要怎麼改？
12. 怎麼改成一次擲兩顆骰子，並顯示兩顆的點數和總和？

(同一則提問同時問了兩個程式，這份紀錄收錄的是回答中「HW1：登入視窗」的部分)

Response:

1. JFrame 是什麼？為什麼我的類別要寫 extends JFrame ？

- 是什麼：JFrame 是 Swing 的「頂層視窗」，就是有標題列、邊框、最小化／最大化／關閉按鈕的那個視窗。按鈕、輸入框這些元件都要放進某個視窗裡才看得到。
- 為什麼要 extends：這是繼承。寫了 extends JFrame，你的類別「就是一個視窗」，可以直接呼叫 setTitle()、setSize()、add() 這些 JFrame 的方法，不用在前面加物件名稱。
- 另一種寫法：不繼承也可以，改成在類別裡建立一個視窗物件（這叫組合）：

```
JFrame frame = new JFrame("登入");
frame.setSize(300, 200);
frame.add(new JButton("登入"));
```

小程式用繼承比較直覺；程式變大之後，很多人反而偏好組合，因為類別不會被 JFrame 一大堆方法綁住。

2. setLayout(null) 是什麼意思？跟 FlowLayout、BorderLayout 有什麼差別？

- `setLayout(null)`：不用任何版面配置管理器，改成「絕對定位」。每個元件的位置和大小都要自己用 `setBounds()` 指定，沒指定的話大小就是 0×0 ，畫面上看不到。
 - 優點：位置完全由你決定，很直覺。
 - 缺點：視窗拉大縮小時元件不會跟著調整，換一台電腦字型大小不同，文字也可能被切掉。
- `FlowLayout`：像打字一樣，元件由左到右排成一列，排不下就換行，每個元件維持它「剛好需要」的大小。`JPanel` 預設就是這個。
- `BorderLayout`：把空間切成東、南、西、北、中五塊，每塊放一個元件。`JFrame` 預設就是這個。
- 另外還有 `GridLayout`（切成大小一樣的格子）等等。

3. `setBounds(x, y, 寬, 高)` 的四個數字分別代表什麼？座標原點在哪裡？

- `x`、`y`：元件左上角的位置。
- 寬、高：元件的寬度和高度。
- 單位都是像素。
- 原點在容器的左上角。以 `JFrame` 來說，元件其實是加在「內容窗格」（content pane）上，所以原點在標題列下方那塊區域的左上角，不是整個視窗的左上角。
- 要注意 `y` 軸是往下增加的，跟數學課的座標方向相反。`y = 25` 的元件會在 `y = 65` 的元件上面。

4. 為什麼 `setVisible(true)` 要放在所有元件都加完之後？

- 呼叫 `setVisible(true)` 時，Swing 會根據當下的元件排好版面，再把畫面畫出來。
- 如果先顯示視窗才 `add()` 元件，新加入的元件不會自動觸發重新排版和重繪，就可能出現「視窗空白，拉一下視窗大小元件才冒出來」的狀況。
- 如果真的要顯示之後才加元件，加完要自己呼叫：

```
revalidate(); // 重新排版
repaint();    // 重新繪製
```

最簡單的作法就是把 `setVisible(true)` 放在建構子的最後一行。

5. Java 的字串為什麼不能用 `==` 比較？`==` 和 `.equals()` 差在哪？

- `String` 是物件。對物件來說，`==` 比的是「兩個變數是不是指向同一個物件」，也就是記憶體位址，不是內容。
- `.equals()` 比的才是「兩個字串的內容是否完全一樣」。
- 容易誤會的地方：直接寫在程式裡的字串（字面值）會被 Java 共用，所以 `"admin" == "admin"` 有時會是 `true`，讓人以為 `==` 能用。但 `getText()` 每次都會建立一個新的字串物件，內容一樣、位址不同，`==` 就變成 `false`。

```
String input = accountField.getText(); // 使用者輸入 admin
input == "admin"           // false (不同物件)
input.equals("admin")      // true (內容相同)
"admin".equals(input)      // true，而且 input 是 null 時也不會出錯
```

- 不分大小寫的比較可以用 `equalsIgnoreCase()`。

6. `JTextField` 和 `JPasswordField` 有什麼不同？為什麼 `getPassword()` 回傳的是 `char[]`，不是 `String`？

- 差別：`JPasswordField` 是 `JTextField` 的子類別，功能幾乎一樣，只是輸入的文字會顯示成圓點（•），旁人看不到。用 `setEchoChar('*')` 可以換成其他符號。
- 為什麼是 `char[]`：
 - `String` 是不可變的，建立之後就改不了，只能等垃圾回收機制哪天把它清掉。這段期間密碼一直以明文留在記憶體裡。
 - `char[]` 用完可以馬上把內容蓋掉，縮短密碼留在記憶體的時間：

```
char[] pwd = passwordField.getPassword();
// .....驗證.....
java.util.Arrays.fill(pwd, '0'); // 用完立刻清掉
```

- `JPasswordField` 的 `getText()` 已經被標成不建議使用（deprecated），就是這個原因。這次作業為了比較方便，先轉成 `String` 再比較，練習用沒關係，但實務上會盡量避免。

7. `addActionListener(e -> ...)` 這種箭頭寫法（lambda）是什麼？不用 lambda 的話要怎麼寫？

- **lambda** 是「一段可以當成參數傳遞的程式碼」，也就是沒有名字的簡短函式。
- 它能用在「只有一個抽象方法的介面」（函式型介面）上。`ActionListener` 只有一個方法 `actionPerformed(ActionEvent e)`，所以可以用 lambda 簡寫。e 就是那個 `ActionEvent` 參數。
- 不用 lambda 有兩種常見寫法：

寫法一：匿名內部類別

```
loginButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        handleLogin();
    }
});
```

寫法二：讓類別自己實作介面

```
public class hw1 extends JFrame implements ActionListener {
    // 建構子裡：loginButton.addActionListener(this);

    @Override
    public void actionPerformed(ActionEvent e) {
        handleLogin();
    }
}
```

三種寫法效果一樣，lambda 只是最短的一種。

8. `System.out.println` 和 `JOptionPane.showMessageDialog` 的差別是什麼？各自適合用在什麼情況？

- **`System.out.println`**：把文字印到**主控台**（終端機、IDE 下方的 Console）。使用 GUI 程式的人通常根本看不到主控台。
 - 適合：開發時除錯、印出變數值、記錄程式執行到哪裡。
- **`JOptionPane.showMessageDialog`**：跳出一個**對話框**給使用者看。對話框是「強制回應」的，要按確定才能繼續操作視窗。
 - 適合：要讓使用者知道的訊息，例如登入成功、登入失敗、輸入格式錯誤。
 - 第四個參數可以換圖示，例如 `INFORMATION_MESSAGE`、`WARNING_MESSAGE`、`ERROR_MESSAGE`。
- `JOptionPane` 還有 `showConfirmDialog`（是／否）和 `showInputDialog`（讓使用者輸入文字）。

9. `setDefaultCloseOperation` 有哪幾種選項？沒有設定的話，按 X 會發生什麼事？

選項	按 X 之後
<code>DO_NOTHING_ON_CLOSE</code>	什麼都不做，通常搭配自己寫的「確定要離開嗎？」
<code>HIDE_ON_CLOSE</code>	把視窗藏起來（預設值）
<code>DISPOSE_ON_CLOSE</code>	關閉並釋放這個視窗；所有視窗都關掉時程式才結束
<code>EXIT_ON_CLOSE</code>	直接結束整個程式

- 沒設定的話就是預設的 `HIDE_ON_CLOSE`：視窗消失了，但程式還在背景執行。從終端機執行時會發現指令一直沒結束，要按 `Ctrl+C` 才停得下來。

10. `SwingUtilities.invokeLater` 是做什麼的？什麼是事件分派執行緒（EDT）？

- **EDT（Event Dispatch Thread，事件分派執行緒）**：Swing 專門用來處理畫面的一條執行緒。重繪畫面、處理按鈕點擊，全部都在這條執行緒上依序進行。
- Swing 的元件**不是執行緒安全**的，所以規則是：建立和修改介面，都要在 EDT 上做。

- `main` 方法跑在另一條「主執行緒」上。`invokeLater` 的作用是把「建立視窗」這件事排進 EDT 的待辦佇列，交給 EDT 去執行：

```
SwingUtilities.invokeLater(hw1::new);    // 在 EDT 上建立視窗
```

- 另一個重點：按鈕的事件處理本身就跑在 EDT 上。如果在裡面做很久的事（例如連網、讀大檔案），EDT 被卡住，整個視窗就會「凍住」不動。這種耗時工作要改用 `SwingWorker` 之類的方式放到背景執行。

11. 為什麼把帳號密碼直接寫在程式碼裡不安全？實務上通常會怎麼處理？

- 不安全的原因：
 - 編譯好的 `.class` 檔可以被反編譯，寫在裡面的字串很容易被翻出來。
 - 程式碼上傳到 GitHub，公開的 **repo** 誰都看得到，密碼等於直接公布。
 - 要換密碼就得改程式、重新編譯、重新發布。
 - 所有使用者共用同一組帳密，沒辦法分別管理。
- 實務上的做法：
 - 帳號資料存在資料庫，密碼**不存明文**，只存加鹽（salt）後的雜湊值，常見演算法有 `bcrypt`、`Argon2`。
 - 登入時把使用者輸入的密碼用同樣方式雜湊，再跟資料庫裡的值比對。
 - 驗證放在**伺服器端**做，不放在使用者電腦上的程式裡。
 - API 金鑰這類機密放在環境變數或設定檔裡，而且不放進版本控制。
- 這次作業把 `admin / 1234` 寫死只是練習用，程式裡的註解也有寫到這點。

12. `getText().trim()` 裡的 `trim()` 有什麼用？如果不加，使用者輸入時會遇到什麼問題？

- `trim()` 會把字串**開頭和結尾**的空白（包含空格、Tab、換行）去掉，中間的空白不會動。

```
" admin ".trim()    // "admin"
"ad min".trim()     // "ad min" (中間不動)
```

- **不加的話**：使用者不小心多打一個空格，或複製貼上時多帶了空白，`"admin "` 跟 `"admin"` 就不相等，明明帳號正確卻一直登入失敗，而且空白肉眼看不出來，很難發現原因。
- 補充一：`trim()` 去不掉**全形空白**（中文輸入法常打出來的那種）。Java 11 之後的 `strip()` 可以處理 Unicode 的各種空白，對中文使用者更保險。
- 補充二：**密碼不要 trim**。空白有可能本來就是密碼的一部分，所以 `hw1` 只對帳號做 `trim()`，密碼保持原樣。